

AI-Powered CV Assistant

Deterministic, explainable candidate ranking — **scores own the rank**, AI explains it.

Instant ranked shortlists

Explainable scores

No external services

Flutter
(M3)

FastAPI

SQLite

Deterministic
scoring

Optional
LLM

Status: MVP implemented & tested — 37/37 backend tests passing · ready for production hardening

Manual screening is slow, inconsistent, and hard to defend

- **Manual triage cost** — reading and comparing resumes consumes 30–40% of recruiter time per role
 - Directly inflates cost-per-hire and time-to-fill
- **Inconsistent screening** — different recruiters weight the same CV differently
 - Decisions are subjective and hard to justify
- **No audit trail** — no record of *why* a candidate was ranked where they were
- **Skill mismatch blindness** — strong but differently-worded experience is easy to overlook

Impact

- Slower hiring
- Lower quality shortlists
- Undefendable decisions
- Missed strong candidates

The opportunity

- One rubric for every candidate
- Same weights, same logic, every time
- Reasoning shown, not hidden

An end-to-end pre-screening pipeline



Describe the job

Paste a JD or upload a file → normalized into structured requirements



Upload CVs

Batch PDF/DOCX/TXT files → each parsed into a structured candidate profile



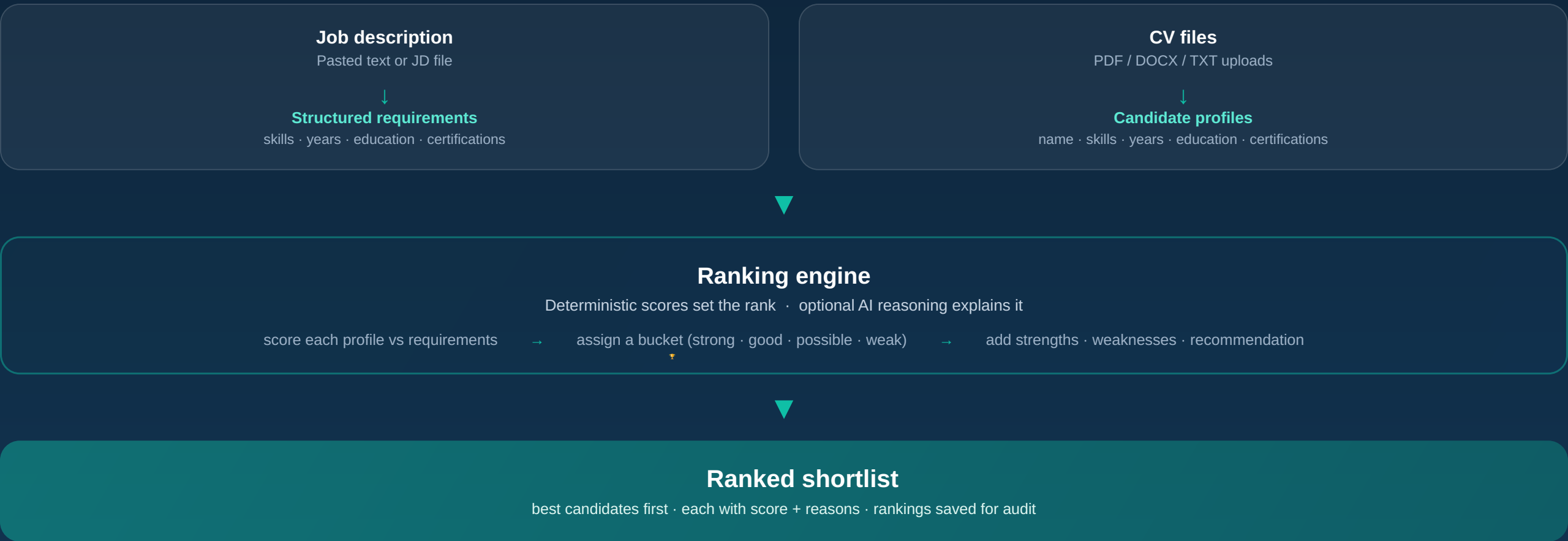
Rank & shortlist

Ranked candidates, best first, each with score + reasoning

Design principles

- **Rules own the rank, AI explains.** Scores come from transparent formulas; the LLM only adds a reasoned narrative.
- **Deterministic fallback.** No LLM configured — or it fails? Full rankings still returned from rules alone.
- **Self-contained.** SQLite + local file storage. No external services required to run.
- **Provider-agnostic.** Any OpenAI-compatible LLM (OpenAI, Gemini, Ollama) selected by config, never code.

How the pieces interact



Proven, swappable, self-contained

- **Backend** — Python FastAPI (single process), async & typed
 - Domain folders: parsers · skills · jd · extraction · ranking
- **Storage** — SQLite + on-disk uploads, zero external services
- **Extraction** — local BERT NER → LLM → deterministic rules fallback chain
- **Skill matching** — 200+ skills, aliases, certifications, open-vocabulary pass
- **LLM** — any OpenAI-compatible endpoint, default Gemini, temperature 0
- **Frontend** — Flutter (Material 3), one codebase for web/iOS/Android
- **Quality** — pytest suite, 37/37 passing; script runner for dev

Key implementation facts

- Ranking weights configurable via environment
- Skill dictionary is a single file to extend
- LLM key optional — everything works without it

No external services

- SQLite + local files
- Runs on a single machine or container
- Deploy anywhere, including on-prem

How we measure success — and what we need

METRICS

- ▶ Pipeline reliability — $\geq 95\%$ uploads parsed & ranked
- ▶ Ranking consistency — reproducible given same input
- ▶ Time-to-shortlist — reduced $\geq 40\%$ vs. manual review
- ▶ Extraction accuracy — verified against golden sample set

TO MOVE FORWARD, WE ASK FOR

- ▶ Approval of this proposal and the Phase 4 roadmap
- ▶ Pilot dataset — 20–30 real sample resumes to validate parsing
- ▶ Team allocation for Phase 4 roles
- ▶ Deployment target — container host / VM / managed PaaS